

Tanishk Arjun Kalaskar

EN23107054

Practicak no 02

Batch: C

```
In [11]: import torch, torch.nn as nn, torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import numpy as np
from sklearn.metrics import classification_report

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([transforms.ToTensor()])
train = datasets.MNIST("./data", train=True, download=True, transform=transform)
test = datasets.MNIST("./data", train=False, download=True, transform=transform)
train_loader = DataLoader(train, 64, shuffle=True)
test_loader = DataLoader(test, 64)

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28,128),
            nn.ReLU(),
            nn.Linear(128,10)
        )
    def forward(self,x): return self.net(x)

model = Model().to(device)
loss_fn = nn.CrossEntropyLoss()
opt = optim.Adam(model.parameters(), lr=0.001)

for _ in range(3):
    for x,y in train_loader:
        x,y = x.to(device), y.to(device)
        opt.zero_grad()
        loss_fn(model(x), y).backward()
        opt.step()

preds, labels = [], []
with torch.no_grad():
    for x,y in test_loader:
        p = model(x.to(device)).argmax(1)
        preds += p.cpu().tolist()
        labels += y.tolist()

print("Accuracy:", np.mean(np.array(preds)==np.array(labels))*100)
print(classification_report(labels, preds))
```

Accuracy: 96.83

	precision	recall	f1-score	support
0	0.97	0.99	0.98	980
1	0.99	0.98	0.99	1135
2	0.97	0.97	0.97	1032
3	0.96	0.97	0.97	1010
4	0.96	0.98	0.97	982
5	0.98	0.95	0.97	892
6	0.97	0.97	0.97	958
7	0.97	0.96	0.97	1028
8	0.94	0.97	0.95	974
9	0.97	0.94	0.96	1009
accuracy			0.97	10000
macro avg	0.97	0.97	0.97	10000
weighted avg	0.97	0.97	0.97	10000

In []: